

Software Lifecycle Management



Fachbereich Elektrotechnik
und Technische Informatik

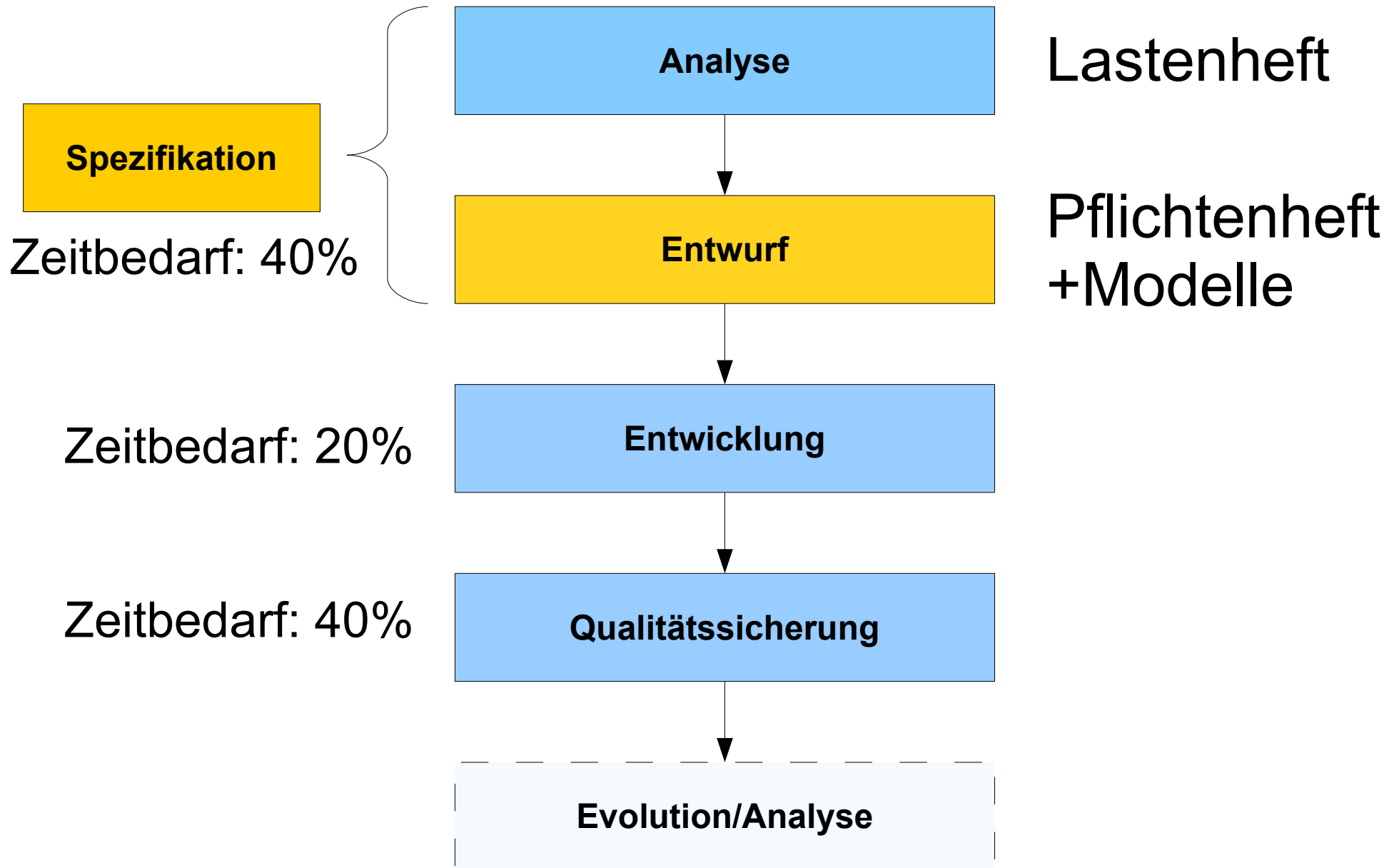
Dr. rer. nat. Nils Beckmann
nils.beckmann@th-owl.de

Entwerfen

Themen

- Requirements Management (REQ)
 - Entwurf
 - Grob-, Fein-
 - Kriterien für einen guten Entwurf
 - Entwurfsmethoden
 - Technische Spezifikation
 - Werkzeuge
 - Software

Erinnerung: Softwareprozessphasen



Phasen des Software-Entwurfs

Innerhalb der Entwurfsphase

Grob-Entwurf



Fein-Entwurf

- Technische Spezifikation
- Pflichtenheft
- Modelle

- Datenstrukturen
- Algorithmen
- Tickets
- ... geht in die Implementierungs-Phase über...

Grob-Entwurf

- Technische Spezifikation
 - Beschreibung der Funktionalen Einheiten und Anforderungen ("Lastenheft") mit Fachbegriffen
 - Wird auch "Pflichtenheft" genannt
- Verschiedene Arten Modelle:
 - u.A. Kontextmodell, Schichtenmodell
 - Software-Architektur
- Spezifikation von Systemkomponenten
 - Zerlegen und Definieren, Beziehungen aufzeigen
- Spezifikation von Schnittschnellen
 - Was kommuniziert womit

Fein-Entwurf

- Entwurf im Detail:
 - Datenstrukturen
 - Algorithmen
 - Klassen
 - u.A.
- Erstellen der Tickets
 - ...für Features, Backlogs, Haupt- und Unteranforderungen, usw.
- *...geht in der Praxis in die Implementierungsphase über...*

Aktivitäten, Ergebnisse, Verantwortungen

- Aktivitäten:
 - Erstellen von Architekturdokumenten, Technische Spezifizierung, Prototyping, Aufgaben definieren/zerlegen/auflisten/zuteilen sowie in Hierarchie bringen
- Ergebnisse:
 - Architekturdokument, Technische Spezifikation (Pflichtenheft), Systembeschreibung, Modelldiagramme, Ticketliste
- Verantwortungen:
 - Software-Architekten, (Software-)Designer, Software-Entwickler, (Programmierer)

Kriterien für einen guten Entwurf

- Es gibt 4 wichtige Kriterien für einen guten Entwurf:
 - A) Korrektheit
 - B) Erweiterbarkeit
 - C) Verständlichkeit
 - D) Wiederverwendbarkeit

A) Korrektheit

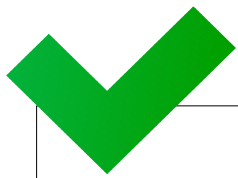
- Umsetzung aller funktionalen Anforderungen – der funktionalen Einheiten – unter Berücksichtigung aller nicht-funktionalen Anforderungen
 - Die Benutzersicht (Lastenheft) wird in eine dementsprechend Technische Sicht umgesetzt
 - Dies kann durch das Kriterium der "Nachverfolgbarkeit" ("Traceability") unterstützt werden (z.B. durch Ticketsysteme wie Redmine)

B) Erweiterbarkeit

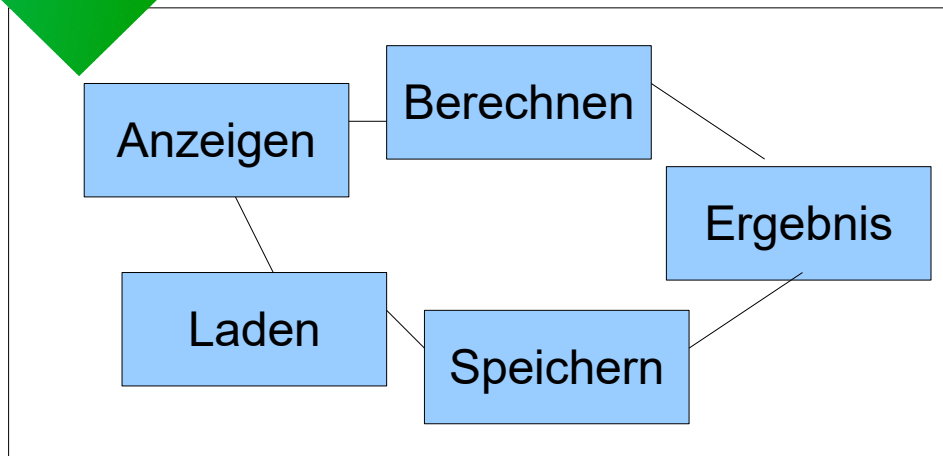
- Die Software soll so spezifiziert werden, dass eine Erweiterbarkeit gut möglich ist:
 - ...während der Implementierungs-Phase
 - ...als auch in der Zukunft (Evolutions-Phase)
- Dies wird gewährleistet durch:
 - i. Starke Kohäsion
 - ii. Schwache Kopplung
 - iii. Schnittstellen
 - iv. Vermeidung von Code-Duplizierung

i. Kohäsion

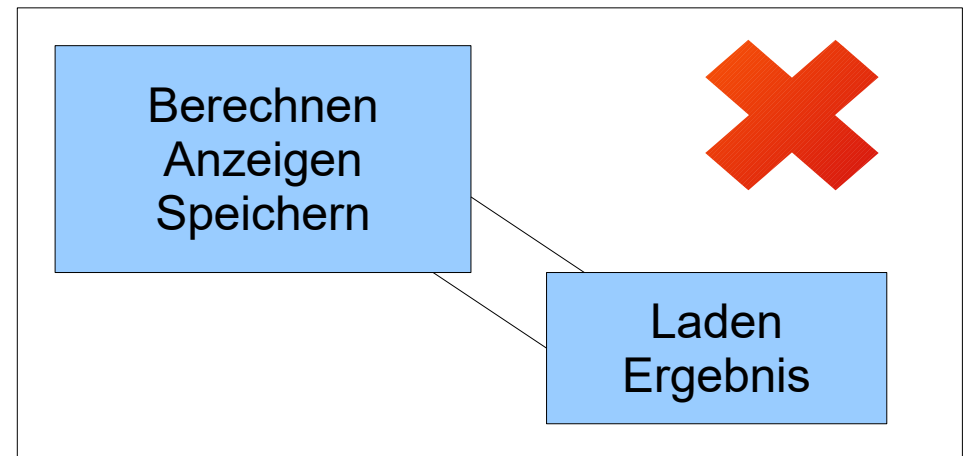
- Maß für die Zusammengehörigkeit/Zusammenhangskraft der in einer Systemkomponente gebündelten Dinge/Eigenschaften/Methoden/etc.
- **Starke Kohäsion** ist das Ziel $\Rightarrow$ Ermöglicht bessere Wiederverwendbarkeit
- Jede logische Einheit (Klasse, Methode, etc.) hat genau eine wohldefinierte Aufgabe



Starke Kohäsion



Schwache Kohäsion



Kohäsion: Beispiel

```
double kreisflaeche (double radius) {  
    return M_PI * pow(radius, 2.0);  
}
```



```
double kreisflaeche (double radius) {  
    double flaeche = M_PI * pow(radius, 2.0);  
    printf("Kreisfläche: %lf", flaeche);  
    return flaeche;  
}
```



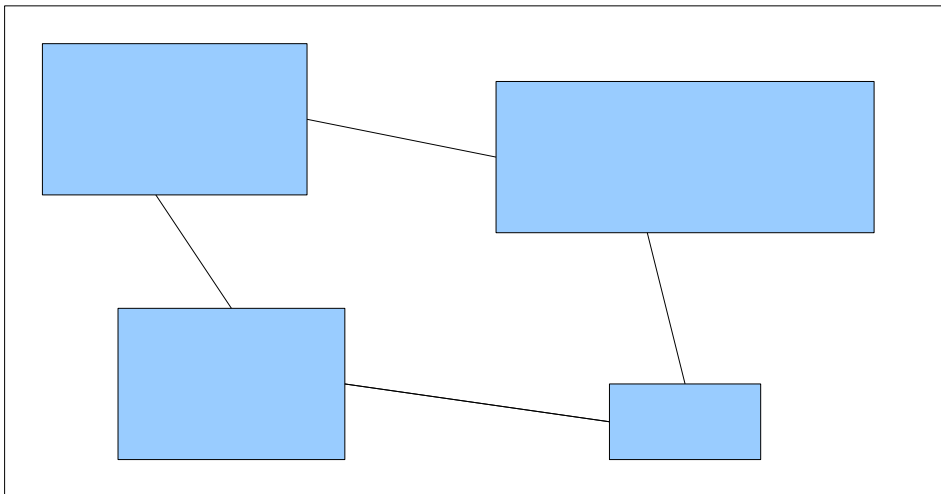
- Im oberen Beispiel hat die Funktion genau eine wohldefinierte Aufgabe: Die Berechnung und Rückgabe der Kreisfläche (= Hohe Kohäsion).
- Im unteren Beispiel wird dies zusätzlich vermischt mit der Ausgabe der Kreisfläche auf der Konsole (= Niedrige Kohäsion).

ii. Kopplung

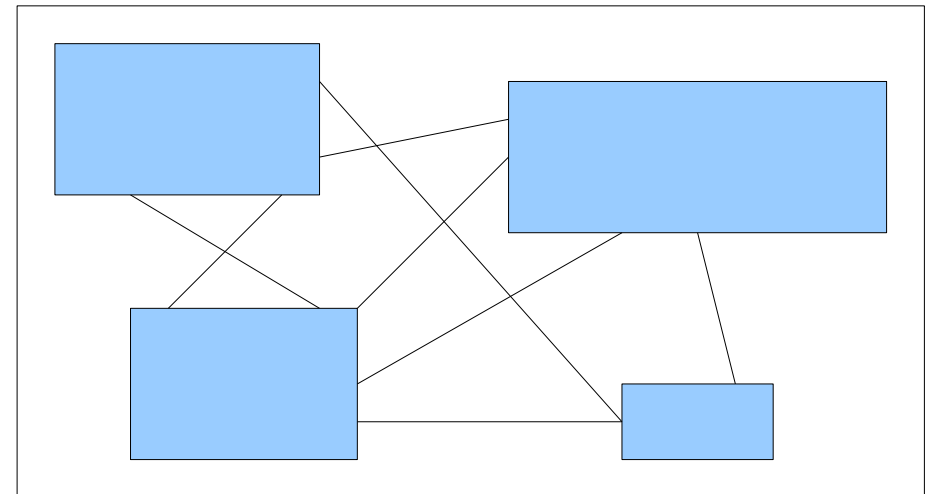
- Grad der Abhängigkeit zwischen den Systemkomponenten
- **Lose Kopplung** ist das Ziel $\Rightarrow$ Systemkomponenten sollen so weit wie möglich unabhängig voneinander sein
- Geeignet dafür ist die Kommunikation über Schnittstellen (*interfaces*)



Lose Kopplung

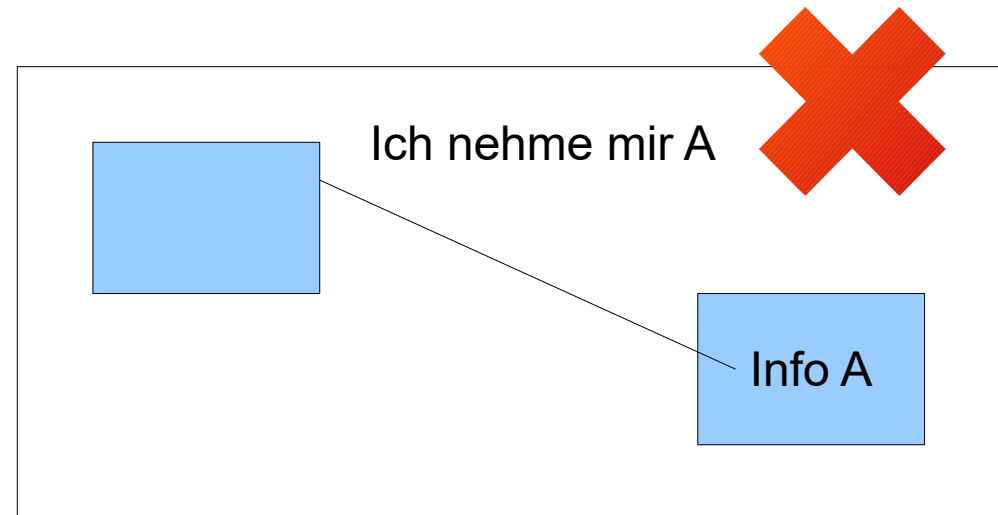
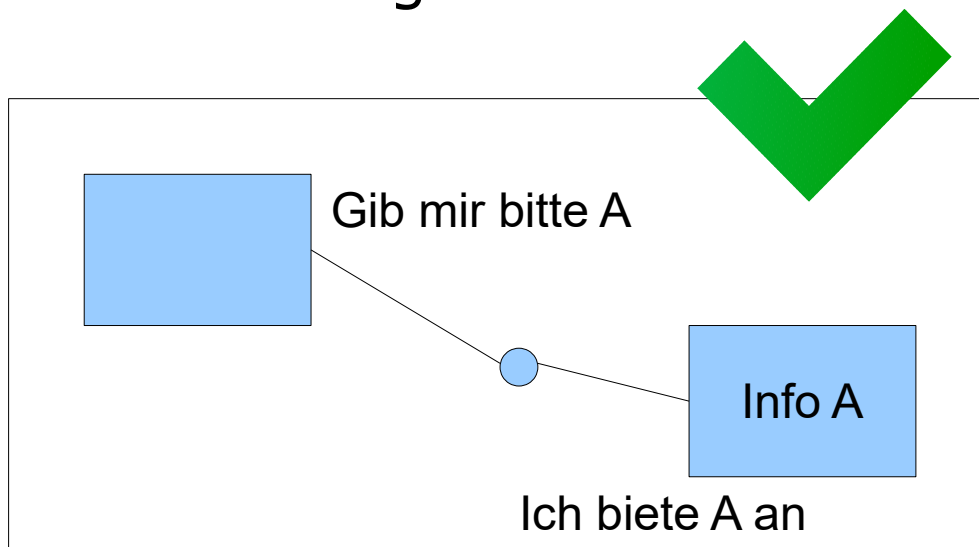


Enge Kopplung



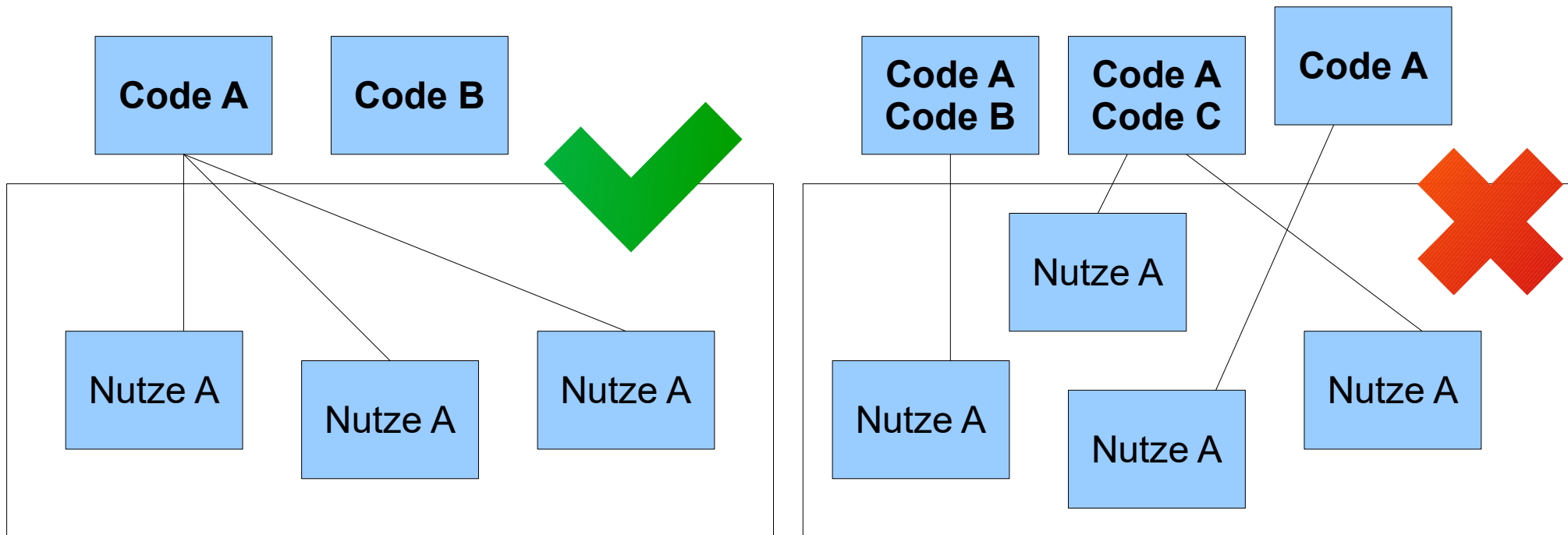
iii. Schnittstellen

- Schnittstellen zu nutzen dient einer "Gesprächskultur" zwischen den Systemkomponenten
- Die anfragende Komponente braucht kein Wissen über die innere Struktur der bereitstellenden Komponente
- Schnittstellen-Änderungen sind ohne Ankündigung problematisch $\Rightarrow$ Meistens nur Erweiterungen aber keine Änderungen an bestehenden Vereinbarungen möglich



iv. Code-Duplizierung

- Ziel ist es, Code-Duplizierung möglichst zu vermeiden
⇒ Module sollten **wiederverwendet** werden können
⇒ Dies Zeit und Aufwand und Kosten
- Klassen können durch **Vererbung** und **Refactoring** (re-)strukturiert werden, um Code-Duplizierung zu reduzieren



Code-Duplizierung: Beispiel



```
void main() {  
    double radius;  
    // ...  
    printf("Kreis A: %lf", M_PI*pow(radius, 2.0));  
    // ...  
    printf("Kreis B: %lf", M_PI*pow(radius, 2.0));  
}
```



```
void main() {  
    double radius;  
    // ...  
    printf("Kreis A: %lf", kreisflaeche(radius));  
    // ...  
    printf("Kreis B: %lf", kreisflaeche(radius));  
}
```

Vermeidung von Code-Duplizierung

- Maßnahmen zur Vermeidung von Code-Duplizierung:
 - Für Klassen:
 - Refactoring
 - Neue Vererbungsstrukturen einführen
 - Für wiederkehrende Code-Blöcke:
 - Methode dafür definieren
 - Ab einer Wiederholung von ≥ 2
 - Prophylaktisch in der Entwurfsphase präzise planen, so dass notwendige Methoden und Klassen bereits vorher identifiziert werden können

C) Verständlichkeit

- **Verständlichkeit** wird erhöht durch:
 - Erstellen und Einhalten von Programmierrichtlinien
 - Quelltext lesbar und leicht verständlich schreiben
 - Lesbare Kommentare nutzen
 - Technische Dokumentation erstellen
 - Tickets mit guten Beschreibungen ausstatten
 - u.A.

D) Wiederverwendbarkeit

- **Wiederverwendbarkeit** wird erhöht durch Starke Kohäsion und Lose Kopplung
 - Erhöht damit die Qualität
 - Weil einzelne Komponenten in verschiedenen Kontexten getestet werden können.
 - Reduziert Redundanz (doppelte Entwicklungen)
 - Reduziert Aufwand, Zeitbedarf und Kosten
 - Erhöht einheitliches Systemverhalten, weil das Verhalten der Systemkomponenten bekannt ist
 - Verringert Code-Duplizierung

Zusammengefasst: Kriterien für einen guten Entwurf

A) Korrektheit

Alle funktionalen Anforderungen sind unter Berücksichtigung der nicht-funktionalen Anforderungen umgesetzt

B) Erweiterbarkeit

Starke Kohäsion, Geringe Kopplung, Schnittstellen verwenden

C) Verständlichkeit

Lesbarkeit von Quelltext, Kommentaren, Dokumentation, Tickets

D) Wiederverwendbarkeit

Modularisierung in Verbindung mit starker Kohäsion, geringer Kopplung, Schnittstellen

Methoden für den Software-Entwurf

- Technische Spezifikation, Pflichtenheft
- Modelle:
 - Kontextmodell
 - Unified Modeling Language (UML)
 - Message Sequence Charts (MSC)
 - Schichtenmodell
 - Ablaufdiagramm
 - Entwurfsmuster
 - usw.

Technische Spezifikation

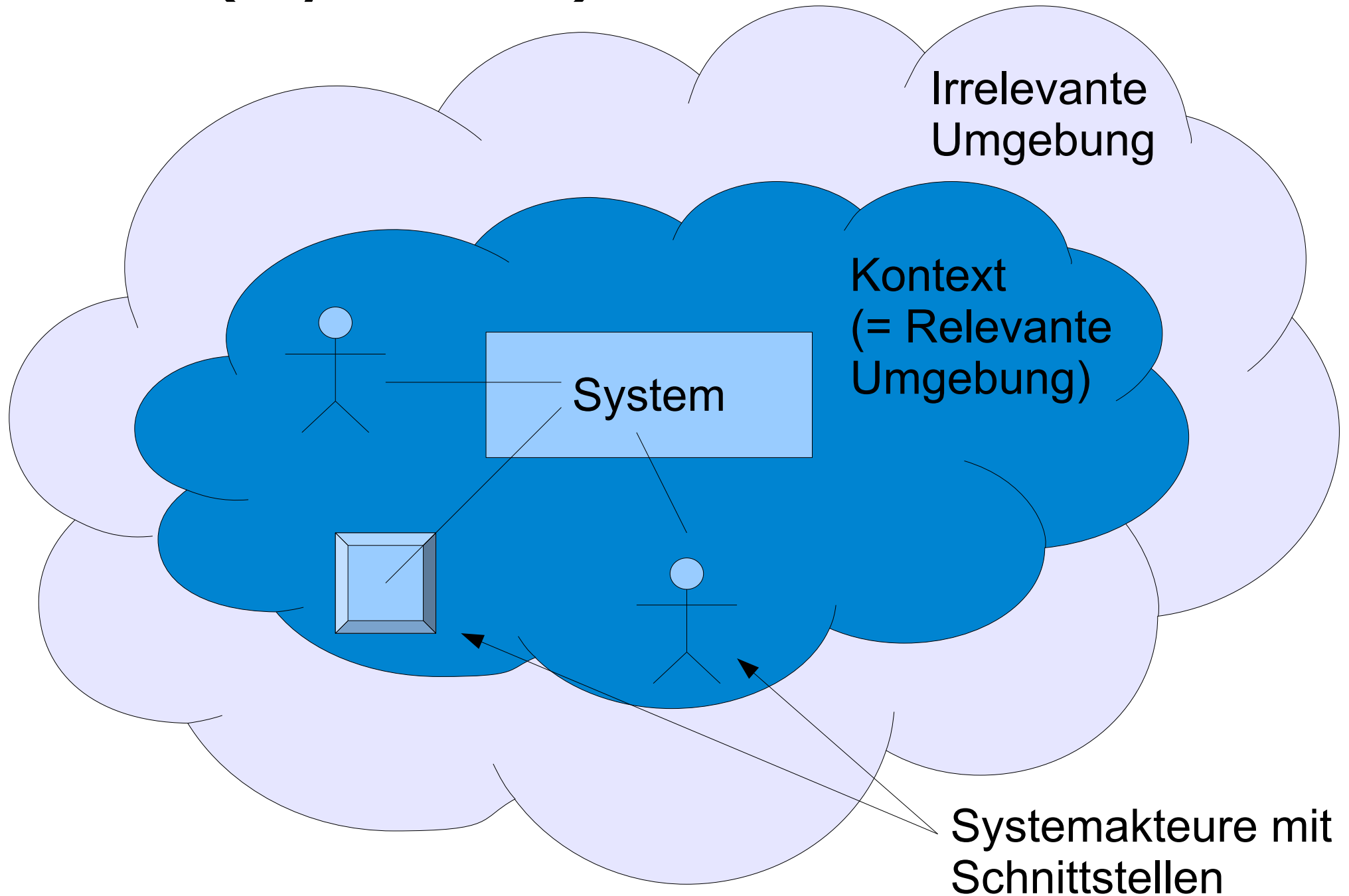
- Das **Pflichtenheft** ist ein geeignetes Werkzeug zur Technischen Spezifikation der Software
 - Es kann aus dem **Lastenheft** entwickelt werden
 - Erinnerung:
 - "Zusammenfassung aller fachlichen Anforderungen an das Softwareprodukt aus Sicht des Auftraggebers inklusive Realisierungsvorgaben aus Sicht des Auftragnehmers."

Strukturen, siehe u.A. Vorlesung, ILIAS-"Vorlagen", „Lehrbuch der Software-Technik“, Balzert, 1996, S. 106 ff., „Software-Engineering“, Sommerville, 2006, S. 169, IEEE/ANSI 830-1998 Standard, usw.

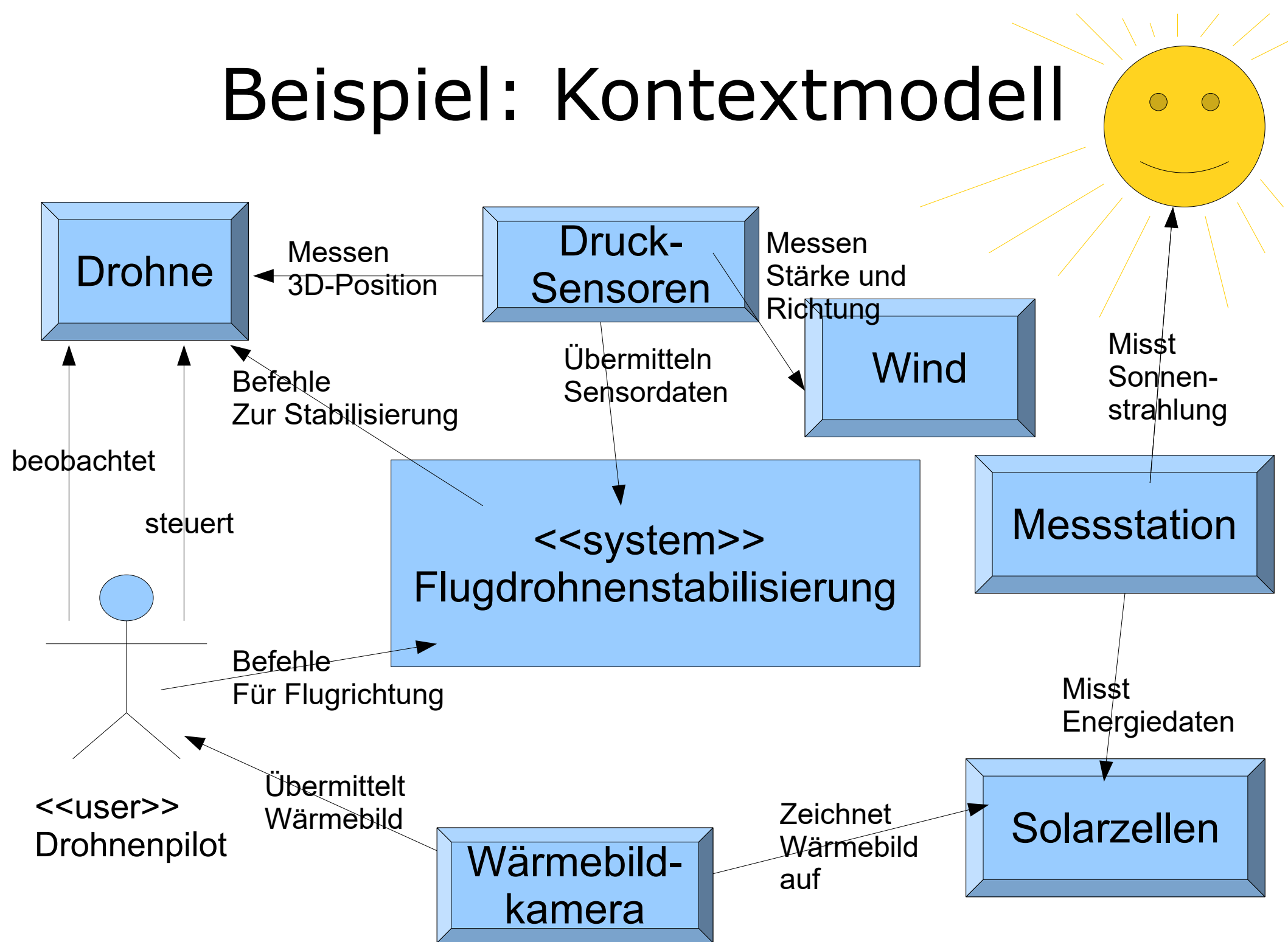
(System-)Kontextmodell

- Gibt einen sehr groben Überblick über die Bausteine und Wechselwirkungen/Schnittstellen des Systems
- Kann später als wichtiger Bezugspunkt für das System immer wieder erinnert werden:
 - Bei der Beschreibung von Anforderungen und System sowie der Implementierung

(System-)Kontextmodell



Beispiel: Kontextmodell



Unified Modeling Language (UML)

- Standardisierte grafische Notation für die technische Spezifikation von Software
- Entwickelt von der "Object Management Group"
- Norm: ISO/IEC 19505
- Verschiedene Diagrammtypen:
 - **Strukturdiagramme** ("Statik" des Software-Systems)
 - Beschreiben die Systemkomponenten und deren Beziehungen
 - **Verhaltensdiagramme** ("Dynamik" des Software-Systems)
 - Beschreiben die Interaktionen zwischen den Systemkomponenten

UML-Diagrammtypen

- Strukturdiagramme ("Statik" des Systems)
 - Klassendiagramm
 - Montagediagramm
 - Komponentendiagramm
 - Verteilungsdiagramm
 - Objektdiagramm
 - Paketdiagramm
 - Profildiagramm
- Verhaltensdiagramme ("Dynamik" des Systems)
 - Anwendungsfalldiagramm
 - Sequenzdiagramm
 - Aktivitätsdiagramm
 - Interaktionsübersichtsdiagramm
 - Kommunikationsdiagramm
 - Zeitverlaufsdiagramm
 - Zustandsdiagramm

Erinnerung: Anwendungsfälle

- **Anwendungsfälle** (*use cases*) beschreiben die von außen wahrgenommenen und verwendbaren Dienstleistungen des Systems
 - Wichtige Ergänzung zu den Anforderungen
 - Werden mit Ablaufbeschreibung versehen und dienen als Ausgangspunkt zum Ableiten der Architektur

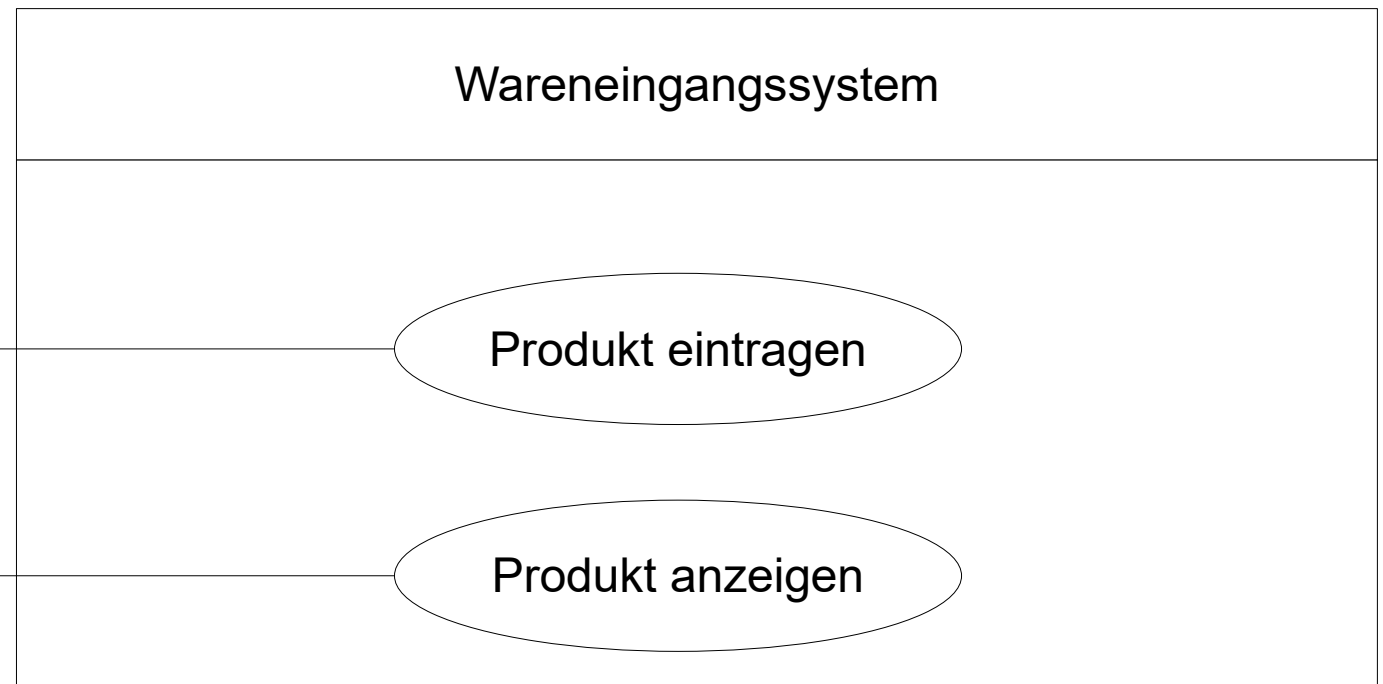
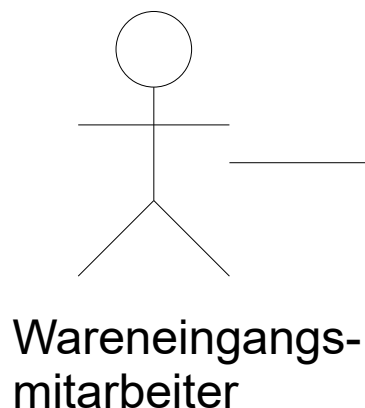
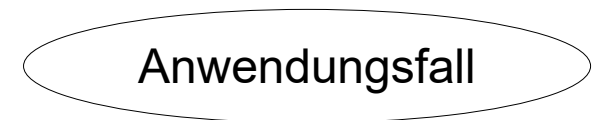
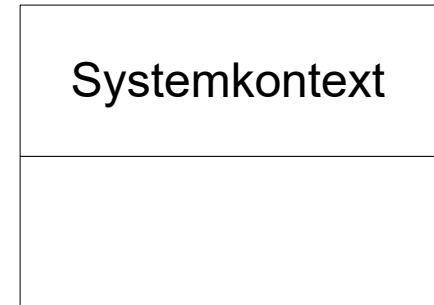
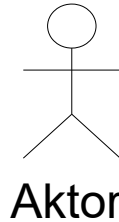
Erinnerung: Anwendungsfall

- Ein **Anwendungsfall** (*use case*) ist eine Zusammenfassung für die Durchführung einer Aktion mit einem bestimmten (Software-)System:
 - Akteur
 - Aktion
 - (Vollständige) Liste der Einzelschritte
 - System
 - Ziel (der Aktion)

UML-Anwendungsfalldiagramm

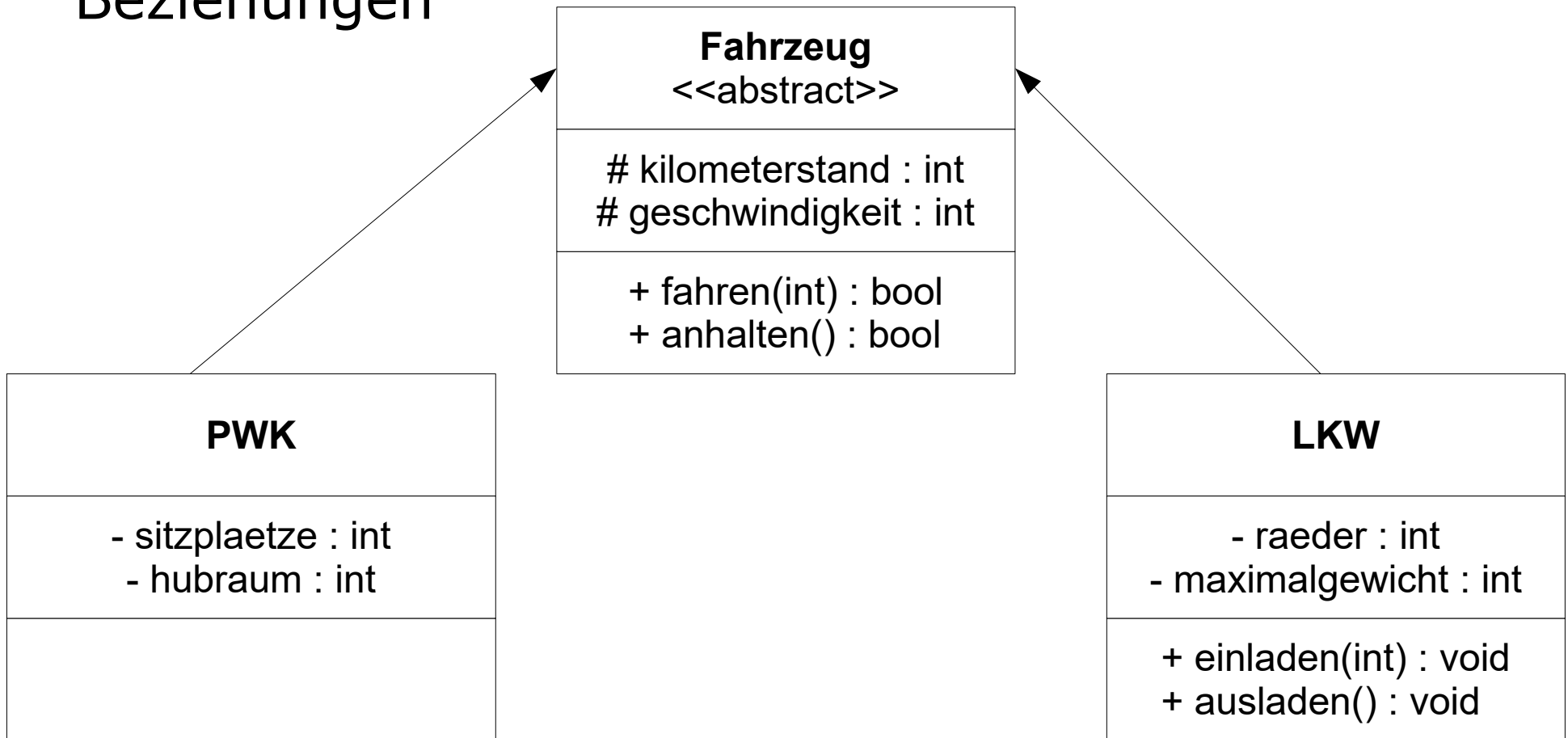
- Bausteine:

- Akteur/Akteur
- (System-)Kontext + (System-)Grenzen
- Anwendungsfälle (Use Cases)



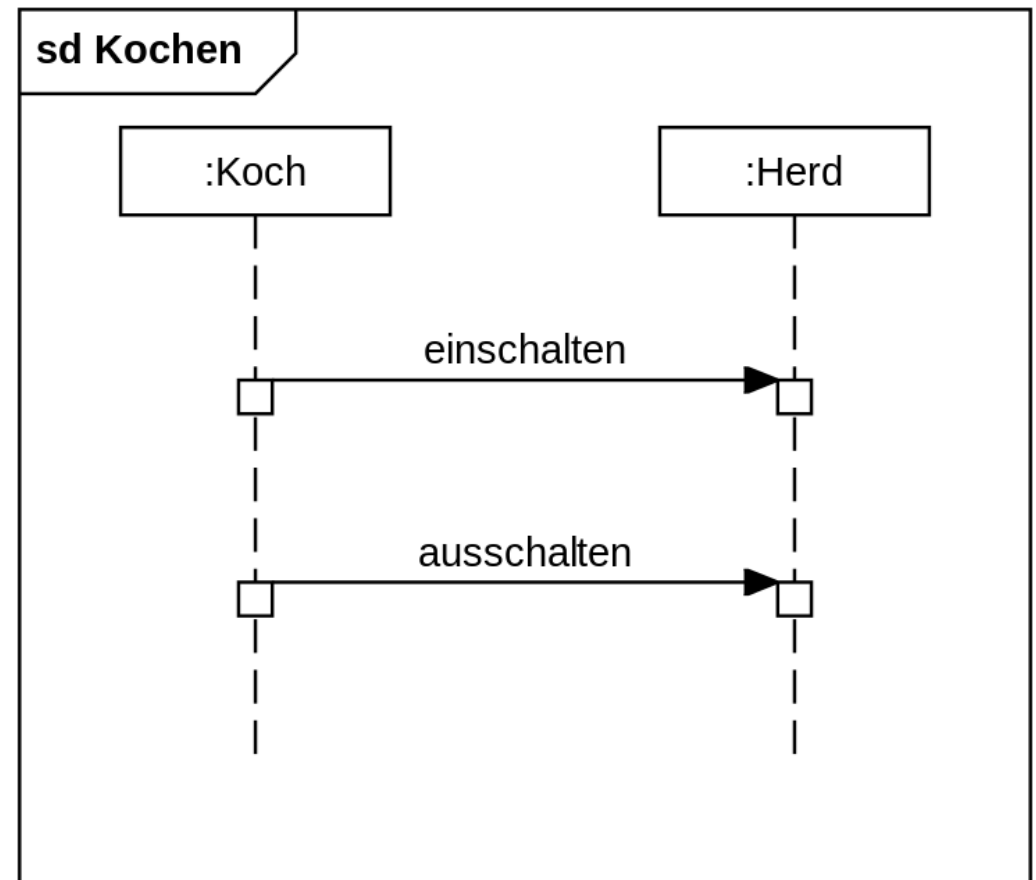
UML-Klassendiagramm

- Neben Anwendungsfalldiagramm das am weitesten verbreitete UML-Diagramm
- Enthält die Klassen/Systemkomponenten und deren Beziehungen



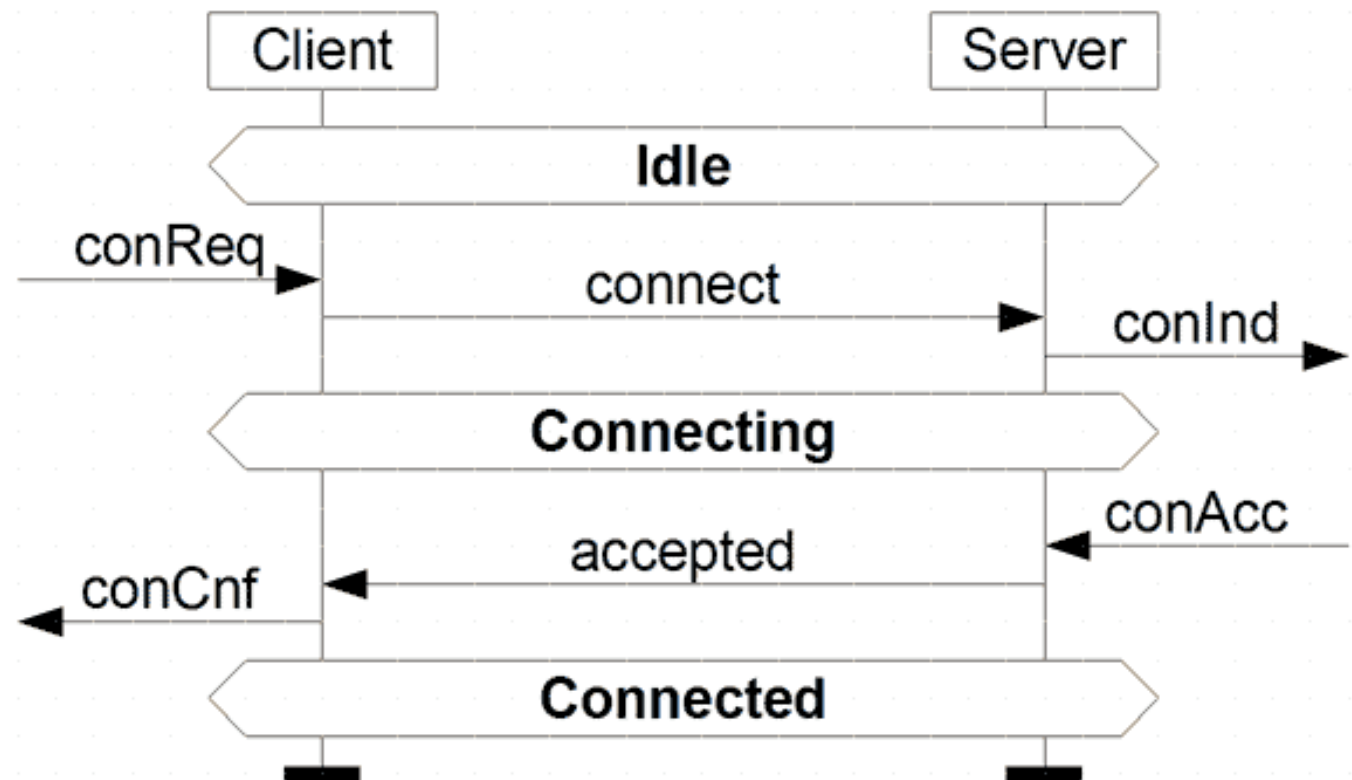
UML-Sequenzdiagramm

- Nachrichten und Antworten entlang einer Zeitachse abbilden
- Interaktion zwischen Objekten
- Einfache Sicht auf den Datenfluss



Message Sequence Chart (MSC)

- Ähnlich dem UML-Sequenzdiagramm



Schichtenmodell

- Modell zur Beschreibung einer Software-Architektur
- Fachliche Trennung grundlegender Aufgaben der Software in Schichten (englisch "tier")
 - $\Rightarrow$ Prinzip der Kohäsion
- Optionale Einschränkung:
 - Übergelagerte Schichte darf untergelagerte Schicht nutzen aber nicht umgekehrt
 - Kennzeichnung durch Pfeile ↓

Schichtenmodell

- Aspekte:
 - Starke Kohäsion
 - Hinzufügen neuer Benutzeroberflächen einfach möglich
 - Web, App/Mobile, Desktop, Server/Client, ...
 - Bessere Aufgabenverteilung bei Änderungen
 - Komplexer in der Umsetzung

Schichtenmodell (Three-Tier-Model)

```
graph TD; A[Präsentationsschicht  
(Benutzeroberfläche)] --> B[Logikschicht  
(Berechnungen, Algorithmen)]; B --> C[Datenhaltungsschicht  
(Dateien, Datenbanken, Datenfelder)];
```

Präsentationsschicht
(Benutzeroberfläche)

Logikschicht
(Berechnungen, Algorithmen)

Datenhaltungsschicht
(Dateien, Datenbanken, Datenfelder)

Schichtenmodell (Two-Tier-Model)

Präsentations- und Logikschicht
(Benutzeroberfläche, Berechnungen, Algorithmen)

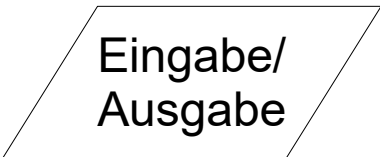


Datenhaltungsschicht
(Dateien, Datenbanken, Datenfelder)

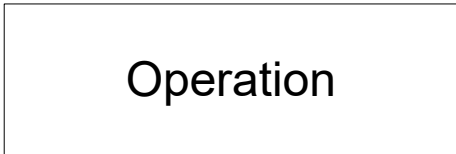
(Programm-)Ablaufdiagramm



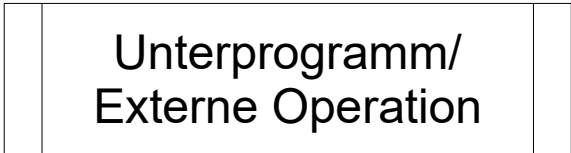
Beginn/Ende
-Zustand



Eingabe/
Ausgabe



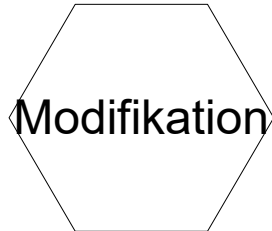
Operation



Unterprogramm/
Externe Operation



Korrekt?



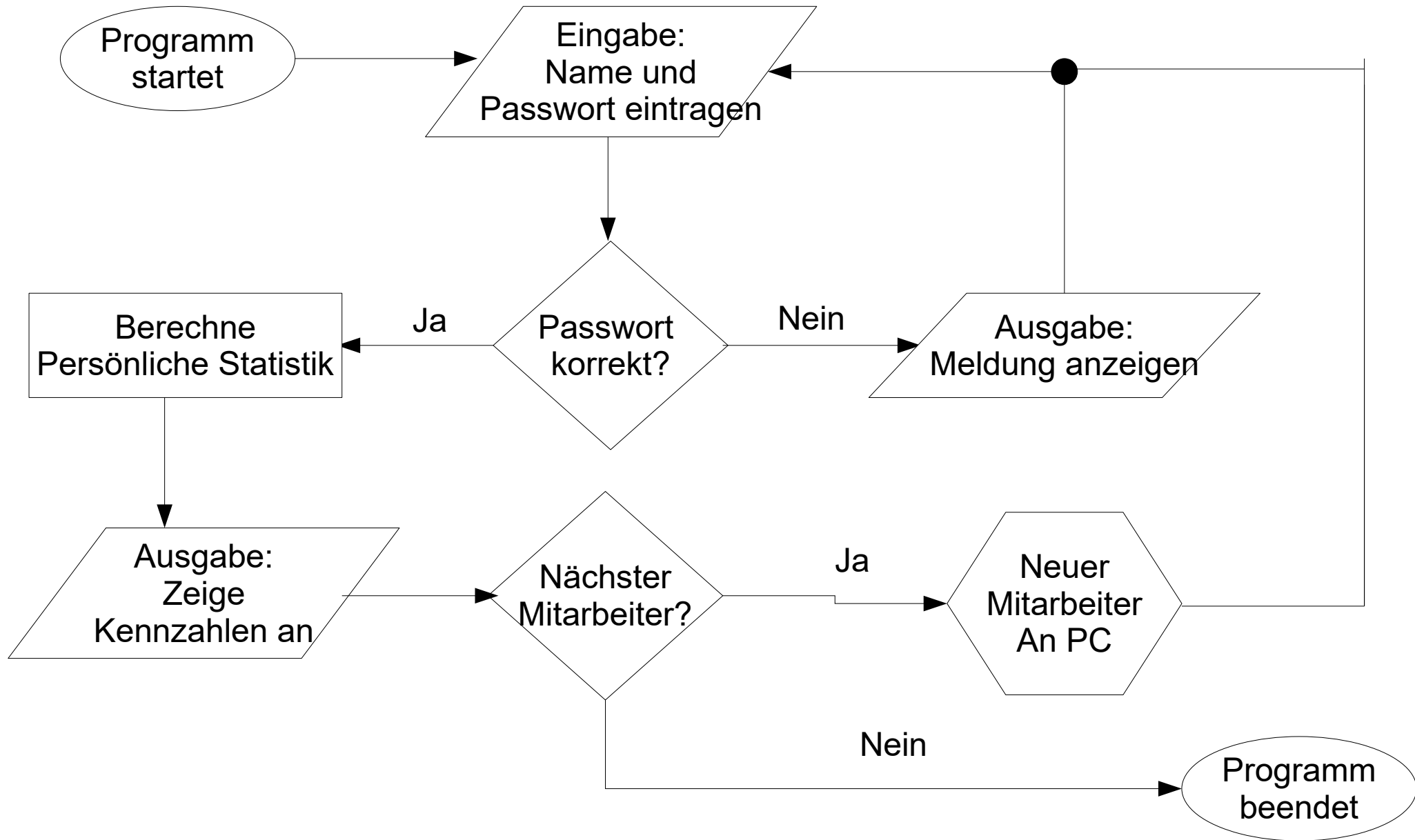
Modifikation

- **OVAL**: Start oder Ende eines Ablaufs
- **PARALLELOGRAMM**: (Externe) Eingabe oder Ausgabe
- **RECHTECK**: Operation innerhalb des Ablaufs
- **RECHTECK (LINIERT)**: Unter-programm oder externe Operation
- **PFEIL**: Verbindungen zwischen den Elementen
- **PUNKT**: Zusammenführen von Verbindungen
- **RAUTE**: Entscheidung oder Verzweigung
- **SECHSECK**: Modifikation von Programm oder Umgebung

z.B. nach DIN 66001, 66262

...und weitere Symbole...

(Programm-)Ablaufdiagramm



Entwurfsmuster

- Erzeugungsmuster (*creational patterns*)
 - z.B. Einzelstück (*singleton*), Fabrikmethode, Prototyp, Erbauer (*builder*), Abstrakte Fabrik, usw.
- Strukturmuster (*structural patterns*)
 - z.B. Adapter, Dekorierer (*decorator*), Privatklassendaten, Kompositum, Fassade, usw.
- Verhaltensmuster (*behavioral patterns*)
 - z.B. Beobachter (*observer*), Iterator, Kommando, NullObjekt, Pipeline, Schablone, Umhüller (*wrapper*), Zustand (*state*), usw.
- Objektrealtionsmuster
- Nachrichtenübermittlungsmuster

Klassiker der Softwaretechnik: Gamma, Helm, Johnson, Vlissides,
„Design Patterns: Elements of Reusable Object-Oriented Software“, 1994 („Gang of Four“)

Tools: UML-Diagramme

- **Frei verfügbar:**

- Argo UML

- <http://argouml.tigris.org/>

- Umlet

- <http://www.umlet.com/>

- u. v. m. (Lange Liste von Tools auf Wikipedia)

- https://en.wikipedia.org/wiki/List_of_Unified_Modeling_Language_tools

- **Kommerziell:**

- Visual Paradigm

- <https://www.visual-paradigm.com/download/community.jsp>

- Microsoft Visio

- <https://products.office.com/de-DE/visio?ms.officeurl=visio>

- u. v. m.

Tools: Ablaufdiagramme

- **Kommerziell:**

- Xmind (<http://www.xmind.net/de/>)
- LucidChart (<https://www.lucidchart.com/>)
- Gliffy (<https://www.gliffy.com/>)

- **Frei verfügbar:**

- Draw.io (<https://www.draw.io/>)
- u. v. m.

- Siehe auch Ordner "Zusatzfolien" im ILIAS

Zusammenfassung

- Der **Entwurf** schließt sich an die **Analyse** an
 - Es entsteht die "Innere Struktur" der Software
 - Er gliedert sich in Grob-Entwurf und Fein-Entwurf
 - Letzterer geht über in die Implementierung
 - Einen guten Entwurf kennezeichnen:
 - A) Korrektheit, B) Erweiterbarkeit,
C) Verständlichkeit, D) Wiederverwendbarkeit
- Es gibt diverse hilfreiche Werkzeuge für den Entwurf
 - Pflichtenheft
 - Modelle
 - Software-Tools

Fragen?